

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO COURSEWORK

DEEP LEARNING AND APPLICATIONS IN COMPUTER VISION

CO5085 - Học kỳ 261 (2026–2027)

E1 · E2 · E3 · A1 · A2

Trường thông tin	Giá trị cần xác nhận
Nhóm	_____
Thành viên / MSSV	_____
Thành viên thứ hai / MSSV	_____
Lớp	_____

TP. Hồ Chí Minh, tháng 10 năm 2026

Bản nháp phương pháp - kết quả full run chưa được điền

Trạng thái và phạm vi báo cáo

Tài liệu này gộp E1–E3 và A1–A2 theo yêu cầu tổ chức của nhóm. Các phương pháp, protocol và định nghĩa metric đã có trong notebook độc lập. Phần kết quả định lượng chính thức chỉ được điền sau khi chạy full, kiểm tra log và khóa protocol đánh giá. Các ô “chưa có” là trạng thái chờ thực nghiệm, không phải giá trị bằng không.

E1–E3 dùng Fashion-MNIST; A1 dùng GTSRB; A2 dùng BDD100K từ archive legacy người dùng cung cấp, với 10 lớp detection và split đã lọc trùng. Code hiện làm việc local; công bố GitHub/Pages chỉ thực hiện sau khi được duyệt. Báo cáo mẫu được dùng để tham chiếu bố cục, font và logo; thông tin cá nhân của mẫu không được tự coi là metadata của bài hiện tại.

Chưa có kết quả chính thức

Chưa có kết quả full run và phân tích của nhóm. Cần bổ sung thông tin thành viên, đóng góp thực tế, khai báo AI, metric test và kết luận từ log thật trước khi nộp.

Contents

Trạng thái và phạm vi báo cáo	i
1 Thiết kế thực nghiệm chung	1
1.1 Mục tiêu và yêu cầu học phần	1
1.2 Tái lập và tính độc lập	1
1.3 Ba chế độ và quản lý evidence	1
1.4 Đánh giá công bằng	1
2 E1: Softmax, MLP và CNN	3
2.1 Dataset và tiền xử lý	3
2.2 Kiến trúc và loss	3
2.3 Training protocol	3
2.4 Kết quả và phân tích chờ hoàn thiện	4
3 E2: Multi-head self-attention và tokenization	5
3.1 Dataset và tokenization	5
3.2 MSA thủ công và reference	5
3.3 Ma trận và protocol	5
3.4 Kết quả và attention map	5
4 E3: LSTM và GRU trên chuỗi ảnh	7
4.1 Biểu diễn và baseline	7
4.2 Recurrent cells và kiểm chứng	7
4.3 Protocol và ma trận	7
4.4 Kết quả và phân tích	7
5 A1: GTSRB, CNN và Transformer	9
5.1 Chọn dataset và split	9
5.2 Preprocessing và kiến trúc	9
5.3 Regime và protocol	9
5.4 Ma trận mở rộng và đánh giá	10
6 A2: Detection với Faster R-CNN và FPN	11

6.1	BDD100K local và protocol dữ liệu	11
6.2	Paper chính: Feature Pyramid Networks	11
6.3	Detector và các thành phần tự hiện thực	12
6.4	Training và kiểm chứng	13
6.5	Metric và giới hạn phiên bản nhãn	13
7	Kết luận và công việc cần hoàn tất	15
7.1	Checklist trước bản nộp	15
7.2	Đóng góp và AI	15
A	Hướng dẫn chạy và lịch nộp	17

List of Figures

2.1	Ảnh mẫu của dataset E1.	3
2.2	Accuracy validation của các full run E1.	4
2.3	Loss và accuracy theo epoch của CNN-B.	4
3.1	Accuracy validation của tám full run E2.	6
3.2	Attention trung bình theo head ở block cuối của model manual đã chọn. . . .	6
4.1	Minh họa thứ tự và feature của chuỗi ảnh.	8
4.2	Accuracy validation của tám full run E3.	8
5.1	EDA trên official train, không dùng test để tune.	9
5.2	Accuracy validation của 14 full run A1.	10
6.1	EDA trên train subset BDD100K của A2.	11
6.2	BDD100K AP50:95 dev của bốn full run A2.	13
6.3	Ví dụ qualitative của D2 trên dev.	14

List of Tables

1.1	Ma trận full run dự kiến	2
2.1	So sánh bốn model E1; bảng full được xuất tự động.	4
3.1	So sánh implementations, tokenizers và head ablation.	5
4.1	So sánh recurrent representation và baseline nội bộ.	7
5.1	So sánh regimes và các thí nghiệm mở rộng A1.	10
5.2	Độ biến thiên ba seed của regime được chọn.	10
6.1	Kết quả tác giả FPN, Table 3: ResNet-50 trên COCO minival; không phải kết quả local [2].	12
6.2	Bốn biến thể kiểm soát cùng RoI head và split	12
6.3	So sánh bốn detection run trên dev.	13
7.1	Khai báo đóng góp cần hoàn thiện	15
A.1	Các deadline, 23:59 GMT+7	17

1. Thiết kế thực nghiệm chung

1.1 Mục tiêu và yêu cầu học phần

E1 so sánh phân loại tuyến tính, MLP và CNN; E2 khảo sát self-attention và tokenization; E3 khảo sát LSTM/GRU trên chuỗi ảnh. A1 so sánh CNN/Transformer, scratch/pretrained và mức freeze. A2 hiện thực detection, so sánh ít nhất hai phương pháp/biến thể và phân tích paper liên quan. Mọi train loop thực hiện forward, loss, backward và optimizer step trực tiếp bằng PyTorch.

1.2 Tái lập và tính độc lập

Mỗi notebook chứa toàn bộ cấu hình, lớp model, xử lý dữ liệu, train/eval và xuất kết quả. Không notebook nào import notebook khác hoặc yêu cầu checkpoint của bài khác. Dataset đã xử lý thành train/val/test là dữ liệu dùng chung. E1–E3 độc lập kiểm tra cùng split seed42 và fingerprint; E3 tự train MLP/CNN để có baseline độc lập.

Kernel dùng Python 3.14.7, PyTorch 2.11.0+cu128, torchvision 0.26.0+cu128; GPU NVIDIA RTX A3000 12GB Laptop. Phiên bản runtime được lưu vào từng config. Không thay thư viện đã có chỉ để khớp một phiên bản cũ trong kế hoạch.

1.3 Ba chế độ và quản lý evidence

Smoke xác nhận kỹ thuật trên subset nhỏ; pilot kiểm tra hành vi học và chi phí; full chạy ma trận chính thức. Ba chế độ có folder, checkpoint và log riêng. Chỉ full xuất bảng/hình vào assets của source LaTeX. Best checkpoint dựa validation/dev; last checkpoint lưu optimizer, scheduler, scaler, RNG, epoch và config hash. Resume bắt đầu sau epoch hoàn tất, không ghép protocol khác vào một run.

1.4 Đánh giá công bằng

Giữ split, normalization, metric và quy tắc chọn best nhất quán trong mỗi so sánh. Test chỉ dùng sau khi khóa quyết định. Loss được tổng hợp theo số mẫu; accumulation chia gradient theo số mẫu thực tế, bao gồm cửa sổ cuối. Accuracy/F1/AP khác đơn vị: không so sánh trực tiếp giá trị giữa classification và detection.

Table 1.1: Ma trận full run dự kiến

Bài	Thí nghiệm	Số run
E1	Linear, MLP, CNN-A/B	4
E2	2 implementations $\times$ 3 tokenizers + head ablation	8
E3	2 recurrent cells $\times$ 3 representations + MLP/CNN	8
A1	8 chính + 2 augmentation + 4 seed repeats	14
A2	C5, FPN, anchors, unfreeze	4
Tổng	Không gồm smoke/pilot	38

2. E1: Softmax, MLP và CNN

2.1 Dataset và tiền xử lý

Fashion-MNIST gồm 60k ảnh train và 10k test, 10 lớp quần áo, grayscale 28×28 [6]. Mỗi lớp giữ 600 ảnh validation bằng seed42, tạo train54k/val6k. Mean/std chỉ tính từ train. Không augmentation để dễ quy chênh lệch về kiến trúc.

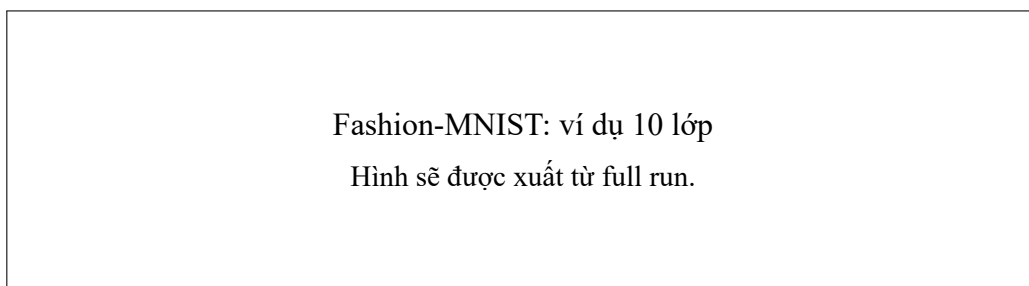


Figure 2.1: Ảnh mẫu của dataset E1.

2.2 Kiến trúc và loss

Linear flatten ảnh thành vector784, head10 tạo logits. MLP dùng 784–256–128–10, ReLU/dropout0.2. CNN-A gồm conv32/64, pooling, FC128. CNN-B gồm conv32/64/128, BN/ReLU/pool, adaptive average pool và dropout0.3. Cross-entropy áp dụng logits:

$$\mathcal{L} = -\frac{1}{N} \sum_i \log \frac{\exp z_{i,y_i}}{\sum_c \exp z_{i,c}}. \quad (2.1)$$

Không softmax trước cross-entropy. Số tham số và thời gian đo từ mỗi model/run.

2.3 Training protocol

AdamW, LR 10^{-3} , weight decay 10^{-4} , batch128, tối đa20 epoch, patience5. Chọn checkpoint bằng validation accuracy, macro-F1 rồi loss khi hòa. Không dùng test để chọn architecture hoặc early stop. Engine tự viết xử lý batch cuối và checkpoint.

2.4 Kết quả và phân tích chờ hoàn thiện

Table 2.1: So sánh bốn model E1; bảng full được xuất tự động.

Nội dung	Trạng thái
Metric full run	Chưa chạy / chờ bổ sung

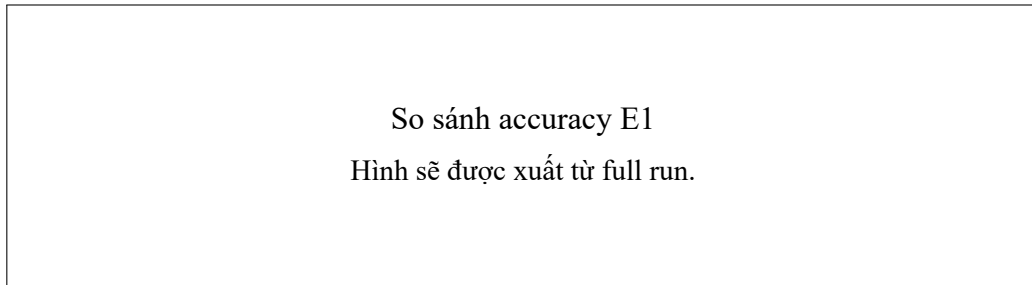


Figure 2.2: Accuracy validation của các full run E1.

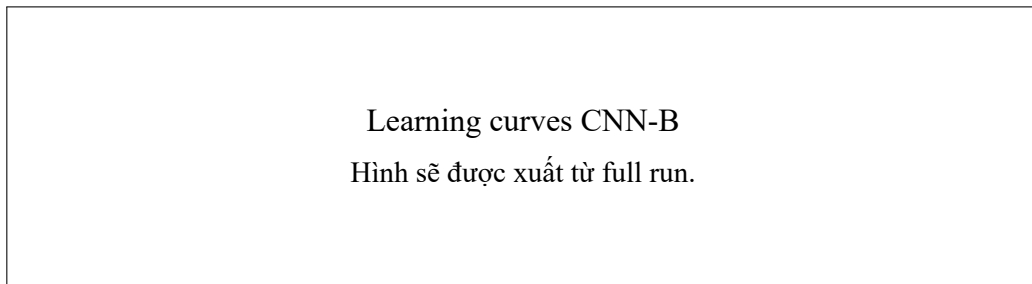


Figure 2.3: Loss và accuracy theo epoch của CNN-B.

Chưa có kết quả chính thức

Điền metric test, confusion matrix và phân tích lỗi sau full run. So sánh inductive bias, capacity và train/validation gap; không kết luận model thắng từ smoke.

Các cặp Shirt/T-shirt/Coat cần xem ảnh thật, kiểm tra texture/outline và độ mơ hồ lớp. Phân tích tối thiểu một lợi ích và một giới hạn của từng family dựa trên evidence.

3. E2: Multi-head self-attention và tokenization

3.1 Dataset và tokenization

Fashion-MNIST dùng split và normalization tự dựng giống protocol E1. Rows tạo 28 token, mỗi token 28 feature. Patch 4×4 tạo 49 token 16 feature theo raster. CNN stem tạo feature $7 \times 7 \times 16$, tức 49 token 16 feature, có tham số học. Khác biệt tokenizer còn bao gồm inductive bias và capacity, cần ghi trong so sánh.

3.2 MSA thủ công và reference

Với $D = 64, H = 4, d_h = 16$, mỗi head thực hiện

$$A_h = \text{softmax}(Q_h K_h^T / \sqrt{d_h}), \quad Y = \text{Concat}(A_1 V_1, \dots, A_H V_H) W_O. \quad (3.1)$$

Manual chỉ dùng Linear, reshape, matmul, softmax; reference dùng PyTorch MHA riêng. Hai block pre-norm, FFN ratio 2, GELU/dropout 0.1, learned position và mean pooling. Kiểm tra copy weight, tắt dropout, so forward/input gradient/projection gradient với tolerance 10^{-5} . Parity này xác nhận phép toán, không đảm bảo hai train run có RNG/dropout và đường tối ưu giống nhau.

3.3 Ma trận và protocol

Sáu run implementations $\times$ tokenizers. Chọn tokenizer manual tốt nhất bằng val, thêm $H=2$ và $H=8$; $H=4$ tái dùng run đã có. AdamW 10^{-3} , WD 10^{-4} , batch 128, tối đa 20 epoch, patience 5. Tổng 8 full run, cùng seed/split.

3.4 Kết quả và attention map

Table 3.1: So sánh implementations, tokenizers và head ablation.

Nội dung	Trạng thái
Metric full run	Chưa chạy / chờ bổ sung

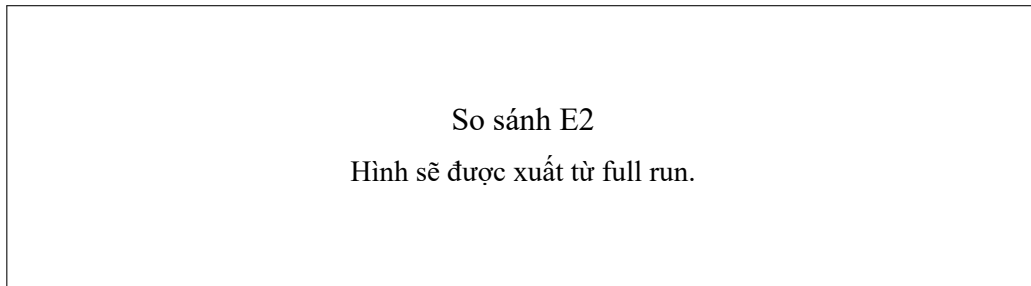


Figure 3.1: Accuracy validation của tám full run E2.

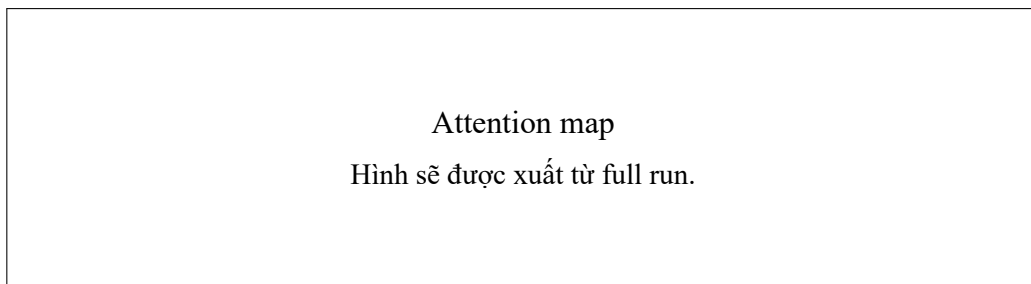


Figure 3.2: Attention trung bình theo head ở block cuối của model manual đã chọn.

Chưa có kết quả chính thức

Điền bảng, curve, test và ảnh lỗi từ full run. Thảo luận sequence length, chi phí attention bậc hai và vai trò position/CNN stem. Attention map mô tả quan hệ token; không tự chứng minh giải thích nhân quả.

4. E3: LSTM và GRU trên chuỗi ảnh

4.1 Biểu diễn và baseline

Rows và columns có $T=28, D=28$; patches 4×4 có $T=49, D=16$. Các ảnh giữ cùng split Fashion-MNIST. Notebook định nghĩa và train MLP/CNN-B riêng, không đọc checkpoint E1. Điều này cho phép chạy từ kernel mới chỉ với raw dataset.

4.2 Recurrent cells và kiểm chứng

LSTM dùng gates input/forget/output, candidate, hidden và cell state:

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \quad h_t = o_t \odot \tanh(c_t). \quad (4.1)$$

GRU dùng reset/update với convention PyTorch:

$$n_t = \tanh(W_{in}x_t + b_{in} + r_t \odot (W_{hn}h_{t-1} + b_{hn})), \quad h_t = (1 - z_t) \odot n_t + z_t \odot h_{t-1}. \quad (4.2)$$

Điểm cần chú ý: reset nhân sau projection hidden, gồm hidden bias. Manual cell được copy weight sang nn.LSTMCell/nn.GRUCell để so forward và gradient tất cả tham số. Model dùng hidden128, một layer, hidden cuối và linear10.

4.3 Protocol và ma trận

LSTM/GRU×ba representation tạo6 run; MLP/CNN-B thêm2. AdamW 10^{-3} , WD 10^{-4} , batch128, max20 epoch, patience5, gradient clipping1 cho recurrent. Baseline giữ protocol riêng tương ứng kiến trúc, không clipping như recurrent.

4.4 Kết quả và phân tích

Table 4.1: So sánh recurrent representation và baseline nội bộ.

Nội dung	Trạng thái
Metric full run	Chưa chạy / chờ bổ sung

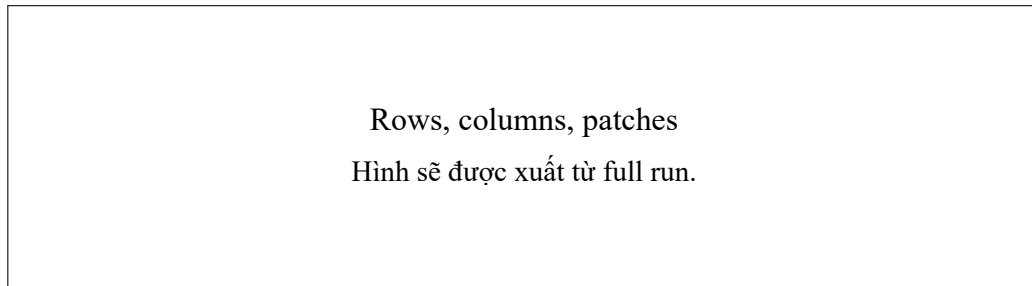


Figure 4.1: Minh họa thứ tự và feature của chuỗi ảnh.

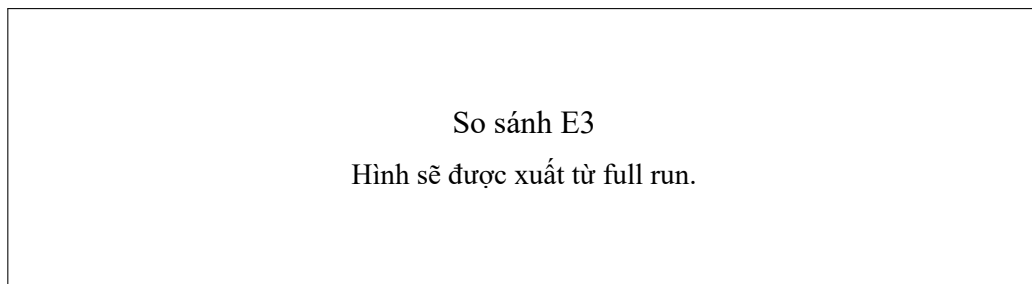


Figure 4.2: Accuracy validation của tám full run E3.

Chưa có kết quả chính thức

Bổ sung kết quả thật, hiệu ứng thứ tự quét, loss gap và lỗi. So sánh chi phí LSTM/GRU, capacity và locality với CNN; hidden state không phải giải thích nhân quả.

5. A1: GTSRB, CNN và Transformer

5.1 Chọn dataset và split

GTSRB có 43 lớp biển giao thông, ảnh đa độ phân giải và các track liên tiếp cùng biển vật lý [4]. Dataset phù hợp A1 vì lớn/khó hơn tập toy chính và có nhiều lớp; không dùng Fashion-MNIST làm bài A1. Train/val tách theo class+track từ filename, giữ khoảng 20% track mỗi lớp cho val, seed 42. Official test giữ đến đánh giá cuối. Không random split các frame của cùng track vì dễ rò rỉ ảnh gần như giống nhau.

Nguồn: kho tác giả ERDA, giữ README gốc và metadata nguồn. Archive được xóa sau khi chia train/val/test; ảnh PPM chuyển PNG lossless và kiểm tra pixel với nguồn. Trang metadata không công bố mã SPDX; không gán giấy phép library cho dữ liệu. Cần kiểm tra điều kiện đi kèm, cite tác giả và không phân phối raw data khi chưa xác nhận quyền.

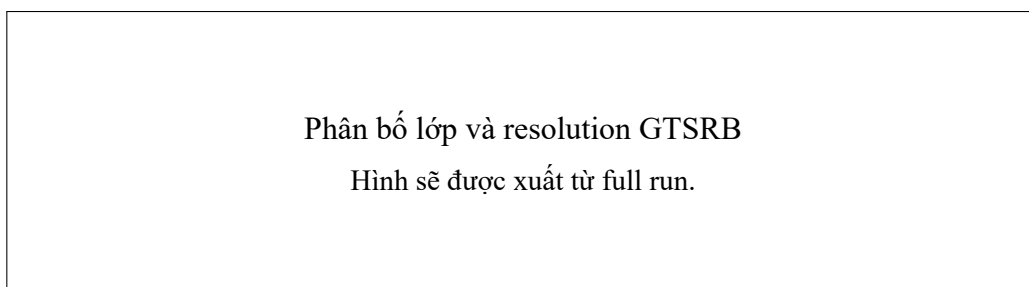


Figure 5.1: EDA trên official train, không dùng test để tune.

5.2 Preprocessing và kiến trúc

RGB 224×224 , normalization ImageNet cho mọi regime. Train augmentation gồm rotate 10 độ, translate 0.05 và color jitter nhẹ; không flip vì có thể đổi ý nghĩa biển. ResNet-18 custom BasicBlock cùng tensor naming torchvision. DeiT-Tiny non-distilled: patch 16, D192, depth 12, heads 3, MLP ratio 4, CLS/position, LayerNorm $\text{eps} 10^{-6}$ [1, 5]. Load tất cả backbone tensors; chỉ thay classification head 43. BN num-batches counter không có trong checkpoint ImageNet cũ được khởi tạo 0; không bỏ qua learned weight hoặc running mean/variance bị thiếu.

5.3 Regime và protocol

Mỗi family chạy scratch, pretrained head-only, partial và full. Partial ResNet mở layer 4/head; partial DeiT mở hai block cuối/norm/head. Frozen BN giữ running buffers và weight bất biến trong train, được kiểm tra bằng assertion.

AdamW WD0.01, microbatch32, accumulation2, effectivebatch64. Scratch $LR3 \cdot 10^{-4}$, max40 epoch; pretrained head 10^{-3} /backbone 10^{-4} , max25 epoch. Warmup3 epoch, cosine, patience8. Chọn bằng val accuracy/F1/loss. Seed43/44 đổi RNG train, giữ split42.

5.4 Ma trận mở rộng và đánh giá

Tám run chính. Với best regime từng family, thêm no-augmentation và hai seed repeats; tổng14. Báo cáo accuracy, macro-F1, params/trainable, thời gian, VRAM và lỗi.

Table 5.1: So sánh regimes và các thí nghiệm mở rộng A1.

Nội dung	Trạng thái
Metric full run	Chưa chạy / chờ bổ sung

Table 5.2: Độ biến thiên ba seed của regime được chọn.

Kết quả	Trạng thái
Mean/std, n=3	Chờ full run

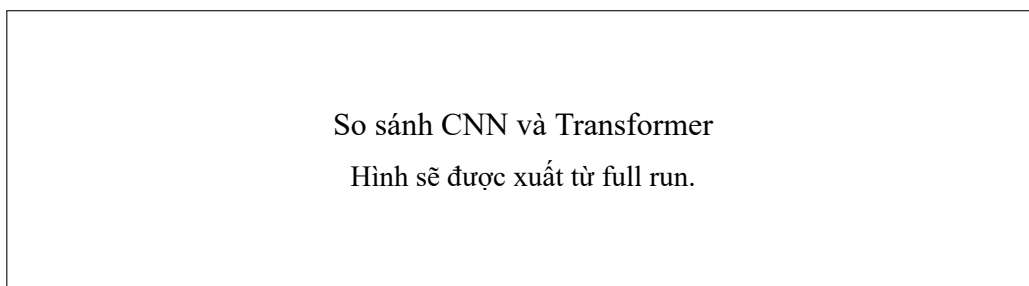


Figure 5.2: Accuracy validation của 14 full run A1.

Chưa có kết quả chính thức

Điền metric official test, seed mean/std, curves và confusion matrices. Giải thích lợi ích pretraining và freeze theo data size; thảo luận no-augmentation và các nhầm lẫn thật. Không khẳng định Transformer thắng/thua trước full run.

6. A2: Detection với Faster R-CNN và FPN

6.1 BDD100K local và protocol dữ liệu

BDD100K có 10 lớp detection trên ảnh giao thông, phù hợp bài phát hiện nhiều đối tượng [7]. Dữ liệu sử dụng là hai archive legacy người dùng cung cấp, đã kiểm tra CRC, SHA256, decode JPEG, parse JSON và đồng bộ 100.000 tên ảnh–nhãn. Quyền dữ liệu và nhãn tuân theo điều kiện BDD100K/BAIR; không đồng nhất với BSD của toolkit. Raw JPEG/JSON giữ nguyên byte trong train/val/test.

Schema name/frames/objects có bbox và polygon. Chỉ box2d dùng cho detection; person/bike/motor ảnh xạ pedestrian/bicycle/motorcycle. Các lớp còn lại là rider, car, truck, bus, train, traffic light và traffic sign; head có 11 logits kể cả background. Tọa độ Scalabel inclusive chuyển bằng cộng 1 vào x2/y2 rồi clamp trong ảnh. Box một pixel được giữ khi còn diện tích; box ngoài mép không còn diện tích sau clamp/float32 bị loại và ghi đầy đủ source box/lý do trong metadata cache. Occluded/truncated vẫn là positive, chỉ crowd/ignored tường minh mới ignore.

Raw split là 70.000/10.000/20.000. Lọc trùng byte bằng SHA256 theo ưu tiên test > val > train; trong split giữ tên nhỏ nhất. Loại 2 ảnh train và 2 dev, gồm một ảnh train trùng test; còn **69.998 train / 9.998 dev / 20.000 test**. Danh sách lọc, subset và fingerprint lưu split.json. Pilot dùng random 2.000/500, seed42; smoke 8/8; full toàn bộ split sạch. Cache NPZ detection được build inline từ JSON có kiểm tra checksum, không nhân bản JPEG hoặc polygon.

Archive này có test JSON nhưng chưa xác nhận tương thích Detection 2020. Vì vậy chỉ báo cáo local legacy test; không nhận đó là official benchmark score. Test chỉ evaluate sau khi khóa protocol, không dùng để chọn anchor hay unfreeze.

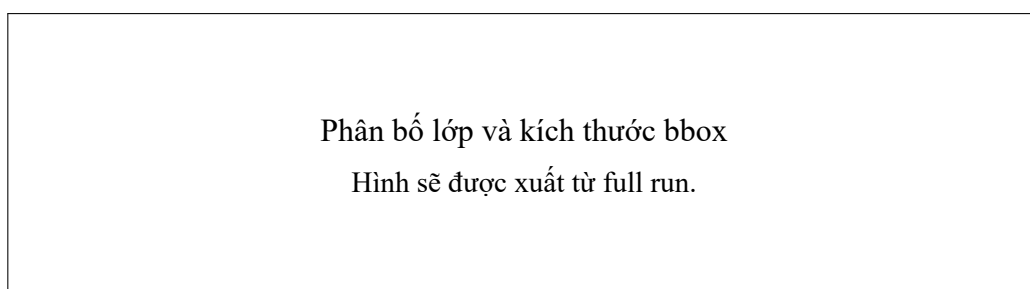


Figure 6.1: EDA trên train subset BDD100K của A2.

6.2 Paper chính: Feature Pyramid Networks

FPN [2] giải quyết chênh lệch semantic giữa feature tầng nông/sâu bằng top-down và lateral connections. Bottom-up ResNet tạo C2–C5. Lateral 1×1 đưa về 256 channel; upsample và

cộng với tầng dưới; conv 3×3 tạo P2–P5. P6 lấy từ pooling P5 cho RPN. RoI được gán tầng theo kích thước:

$$k = \left\lfloor 4 + \log_2 \frac{\sqrt{wh}}{224} \right\rfloor, \quad k \in [2, 5]. \quad (6.1)$$

Faster R-CNN [3] là nền tảng: RPN tạo proposals, RoI head dự đoán lớp và offset bbox. Paper cần được đọc theo phương pháp, thí nghiệm, ablation và giới hạn; không chỉ chép abstract. Bài local kiểm tra nguyên lý feature đa tỉ lệ trên BDD100K, không nhận tái tạo số COCO hoặc mọi chi tiết training của paper.

Table 6.1: Kết quả tác giả FPN, Table 3: ResNet-50 trên COCO minival; không phải kết quả local [2].

Model trong paper	AP	AP50
C4 baseline	31.6	53.1
C5 baseline	28.0	51.7
FPN	33.9	56.9

Các ablation của tác giả bỏ top-down/lateral để kiểm tra vai trò từng nhánh. BDD100K local không tái tạo các ablation này; D3/D4 khảo sát anchor và mức fine-tune.

6.3 Detector và các thành phần tự hiện thực

ResNet50 Bottleneck custom load ImageNet, FrozenBatchNorm. RPN gồm shared 3×3 , objectness và bbox delta, anchor ratios $0.5/1/2$. Matching positive/negative tại IoU $0.7/0.3$ và low-quality match, sample256 positive ratio0.5. Proposals dùng NMS0.7. RoIAlign 7×7 , hai FC1024, classification11 (gồm background), class-specific bbox. ROI sample512 positive ratio0.25, IoU0.5. Box coding và crowd ignore nằm trong notebook. Train có bốn loss RPN objectness/box và ROI classification/box.

Table 6.2: Bốn biến thể kiểm soát cùng RoI head và split

Run	Feature / anchors	Backbone trainable
D1	C5 projected256, size256	C4/C5
D2	FPN, sizes32–512	C4/C5
D3	FPN, sizes16–256	C4/C5
D4	FPN, sizes32–512	Tất cả convolution

C5 là baseline đã chọn, khác C4 baseline trong paper. RoIAlign thay RoIPool và BDD100K khác COCO, vì vậy ghi rõ đây là tái hiện có điều chỉnh. Ngoài feature pyramid, D1/D2 còn khác độ phủ anchor (size256 so với32–512). Do đó D2-D1 là so sánh hai hệ thống; không quy toàn bộ chênh lệch AP cho riêng top-down/lateral. D3-D2 và D4-D2 là các ablation trong family FPN.

6.4 Training và kiểm chứng

SGD momentum0.9, WD 10^{-4} , head LR0.005/backbone0.0005, warmup500 updates, 12 epoch, milestones8/11; shortside640 max1280, flip0.5, micro2 accum2, AMP, clip10. Nếu OOM dùng micro1 accum4, giữ resolution/effectivebatch cho mọi variant. Best theo dev AP50:95; checkpoint nằm trong namespace bdd100k_legacy, tách VOC cũ. Checkpoint lưu toàn bộ trạng thái để resume từ epoch hoàn tất.

Kiểm tra box encode/decode, empty targets, crowd ignore, odd FPN dimensions, AP perfect/false-positive/empty, ánh xạ lớp và chuyển tọa độ inclusive/boundary. Forward/backward/inference chạy cả D1–D4; smoke technical dùng resolution nhỏ, không dùng số smoke để đánh giá giả thuyết FPN.

6.5 Metric và giới hạn phiên bản nhãn

Metric chính là COCO-style AP50:95 bằng pycocotools: IoU0.50–0.95 bước0.05, 101 recall points, maxDets1/10/100. Kèm AP50, AP75, small/medium/large, AR100 và per-class AP/PR tại IoU0.50. Lớp không có GT trong subset ghi null, không gán AP bằng không và không đưa vào mean. Aggregate COCO bằng -1 nghĩa không có GT hợp lệ ở nhóm đó. Không dùng VOC11-point trong A2. Threshold eval0.001, NMS0.5, max100 detections/image; threshold hiển thị0.3 chỉ vẽ hình. Metric này đo local legacy labels, không đồng nhất với benchmark Detection 2020 hoặc số COCO trong paper FPN. FPS ghi batch/GPU và gồm H2D/resize/model/NMS, không gồm disk/COCOeval.

Table 6.3: So sánh bốn detection run trên dev.

Nội dung	Trạng thái
Metric full run	Chưa chạy / chờ bổ sung

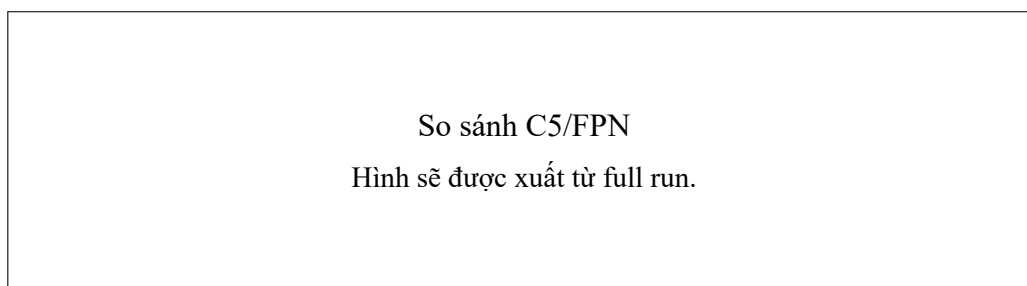


Figure 6.2: BDD100K AP50:95 dev của bốn full run A2.

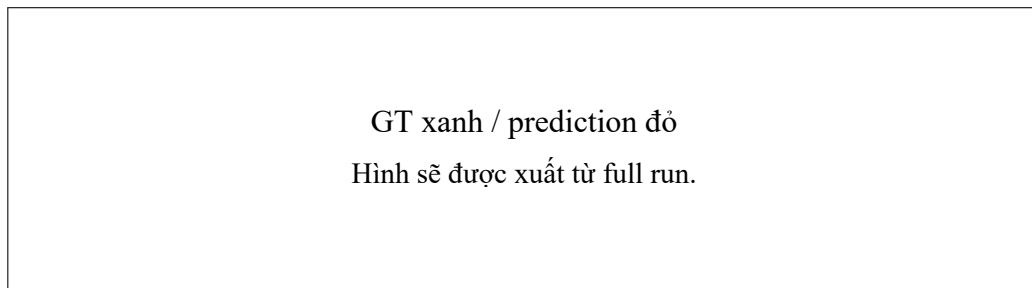


Figure 6.3: Ví dụ qualitative của D2 trên dev.

Chưa có kết quả chính thức

Điền local legacy test, per-class AP, PR curves, AP theo size, FPS và VRAM. Phân tích D2-D1, D3-D2, D4-D2; xem false positive/false negative và lỗi nhỏ/occlusion. Không dùng test để sửa anchor hay chọn unfreeze sau khi đã xem kết quả.

7. Kết luận và công việc cần hoàn tất

Pipeline local đã được tổ chức để thực hiện các thí nghiệm có thể tái lập. Hiệu năng chính thức và ưu/nhược điểm thực nghiệm phải được kết luận sau full run. Không suy ra chất lượng mô hình từ việc forward/backward chạy đúng hoặc từ smoke accuracy.

7.1 Checklist trước bản nộp

Đối chiếu fingerprint split; kiểm tra best/last và config; khóa test protocol; chạy full; đưa log, bảng và hình thật vào báo cáo; viết phân tích lỗi và giới hạn; kiểm tra PDF không tràn trang, citation và link hoạt động; xác nhận metadata và đóng góp.

7.2 Đóng góp và AI

Theo mục 1.6 của đề exercise, nhóm cần hiểu, kiểm chứng và tự đóng góp đáng kể vào code/thực nghiệm/phân tích. Không nộp sản phẩm chủ yếu do AI tạo. Điền bảng dưới bằng đóng góp thực tế, không ghi tên hoặc nhiệm vụ giả định.

Table 7.1: Khai báo đóng góp cần hoàn thiện

Thành phần	Nội dung thực tế
Thành viên	Chưa xác nhận
Code/thực nghiệm	Chưa khai báo đóng góp thực tế
Phân tích/báo cáo	Chưa khai báo đóng góp thực tế
AI hỗ trợ	Hỗ trợ tổ chức notebook, hiện thực ban đầu, kiểm tra và soạn bản nháp; nhóm phải ghi rõ phần đã kiểm chứng/sửa đổi và phần tự thực hiện.

Bibliography

- [1] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016. URL https://openaccess.thecvf.com/content_cvpr_2016/html/He_Deep_Residual_Learning_CVPR_2016_paper.html.
- [2] Tsung-Yi Lin, Piotr Dollar, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. Feature pyramid networks for object detection. In *CVPR*, 2017. URL https://openaccess.thecvf.com/content_cvpr_2017/html/Lin_Feature_Pyramid_Networks_CVPR_2017_paper.html.
- [3] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. In *NeurIPS*, 2015. URL <https://arxiv.org/abs/1506.01497>.
- [4] Johannes Stallkamp, Marc Schlipsing, Jan Salmen, and Christian Igel. Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition. *Neural Networks*, 32:323–332, 2012. doi: 10.1016/j.neunet.2012.02.016.
- [5] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Herve Jegou. Training data-efficient image transformers and distillation through attention. In *ICML*, 2021. URL <https://proceedings.mlr.press/v139/touvron21a.html>.
- [6] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *arXiv:1708.07747*, 2017. URL <https://arxiv.org/abs/1708.07747>.
- [7] Fisher Yu, Haofeng Chen, Xin Wang, Wenqi Xian, Yingying Chen, Fangchen Liu, Vashisht Madhavan, and Trevor Darrell. Bdd100k: A diverse driving dataset for heterogeneous multitask learning. In *CVPR*, pages 2636–2645, 2020. URL https://openaccess.thecvf.com/content_CVPR_2020/html/Yu_BDD100K_A_Diverse_Driving_Dataset_for_Heterogeneous_Multitask_Learning_CVPR_2020_paper.html.

A. Hướng dẫn chạy và lịch nộp

README tại root là hướng dẫn vận hành. Mở notebook bằng kernel Python 3.14 (CO5085), chạy Restart Kernel + Run All; đổi MODE sang full khi đã kiểm tra pilot. Dataset giữ trong datasets; kết quả mỗi bài giữ trong outputs; assets chỉ phục vụ source LaTeX.

Table A.1: Các deadline, 23:59 GMT+7

Ngày	Mốc
19/10/2026	GitHub Pages
26/10/2026	A1.1
02/11/2026	E1, E2, A2.1
09/11/2026	E3, A1.2
16/11/2026	A2.2

Giới hạn trễ theo đề: trừ 20% điểm milestone mỗi tuần, tối đa 7 ngày. Việc công bố remote cần được duyệt trước các mốc phụ thuộc GitHub.